

Lab 1: จำแนกประเภทและออกแบบการจัดเก็บข้อมูล

วัตถุประสงค์: จำแนกประเภทข้อมูล (Raw / Structured / Semi-structured / Unstructured)

ของข้อมูลแต่ละแหล่งในมหาวิทยาลัย และเลือกแหล่งจัดเก็บที่เหมาะสม

ลำดับ / แหล่งข้อมูล	ประเภทข้อมูล	รูปแบบไฟล์ / Schema	แหล่งจัดเก็บที่เหมาะสม
1. ตารางข้อมูลนิสิตในไฟล์ CSV	Structured Data	CSV แถว/คอลัมน์ตายตัว มี schema ชัดเจน เช่น student_id, name, faculty, gpa	Relational Database (เช่น MySQL/PostgreSQL) เพื่อรองรับการ query และ join กับตารางอื่น
2. ข้อมูลจากเซนเซอร์อุณหภูมิ (JSON)	Semi-structured Data	JSON แบบ key-value เช่น {"device_id": "...", "timestamp": "...", "temp_c": "..."} schema ยืดหยุ่น เปลี่ยนแปลงได้	Data Lake หรือ Time-series Database (เช่น InfluxDB) เนื่องจากข้อมูลไหลเข้าต่อเนื่องปริมาณมาก
3. ภาพจากกล้องวงจรปิด	Unstructured Data	ไฟล์ภาพ/วิดีโอ (JPG, MP4) ไม่มี schema ตายตัว เป็น binary object	Object Storage (เช่น Amazon S3, MinIO) เหมาะกับไฟล์ขนาดใหญ่จำนวนมาก
4. ไฟล์ PDF ใบสมัครทุน	Unstructured Data (ถึงมีโครงสร้างถ้ามี form field)	ไฟล์ PDF เนื้อหาผสมข้อความ/รูปภาพ ไม่ใช่ตารางเชิงสัมพันธ์	Object Storage ร่วมกับ Document Management System และเก็บ metadata (เลขที่ใบสมัคร ฯลฯ) แยกไว้ใน Relational DB
5. Log การใช้งานระบบ LMS	Semi-structured Data	Text/JSON log ต่อบรรทัด เช่น timestamp, user_id, action, ip มีโครงสร้างคร่าว ๆ แต่ไม่ตายตัว	Data Lake สำหรับเก็บ log ดิบ แล้วประมวลผลสรุปเข้าสู่ Data Warehouse เพื่อวิเคราะห์พฤติกรรมผู้ใช้
6. ตารางผลการลงทะเบียนเรียนในฐานข้อมูล MySQL	Structured Data	ตารางเชิงสัมพันธ์ มี schema และ constraint ชัดเจน (student_id, course_id, semester, grade)	Relational Database (MySQL) สำหรับงาน transaction และ Data Warehouse สำหรับรายงาน/วิเคราะห์ย้อนหลัง
7. ไฟล์เสียงจากศูนย์รับแจ้งปัญหา	Unstructured Data	ไฟล์เสียง (WAV/MP3) ไม่มีโครงสร้างข้อมูล เป็น binary object	Object Storage พร้อมเก็บ metadata การโทร (วันที่ ผู้รับสาย ระยะเวลา) แยกไว้ใน Relational DB
8. ข้อมูลดิบจากเครื่องอ่านบัตรที่ยังไม่ได้ตรวจสอบ	Raw Data (ก่อนจำแนก มักอยู่ในลักษณะ Semi-structured)	ไฟล์ log/CSV ดิบจากอุปกรณ์ ยังไม่ได้ตรวจสอบความถูกต้อง อาจมีค่าผิดพลาด/ซ้ำ/ว่าง	Data Lake (Staging Area) เก็บข้อมูลดิบทั้งหมดก่อน ทำ Data Cleaning/ETL แล้วจึงย้ายเข้า Data Warehouse

สรุปหลักการเลือกแหล่งจัดเก็บ

- Relational Database เหมาะกับข้อมูลที่มี schema ตายตัว ต้องการ ACID transaction และการ query เชิงสัมพันธ์ (ข้อมูลนิสิต, ผลการลงทะเบียน)
- Data Lake เหมาะกับข้อมูล semi-structured/raw ปริมาณมาก ที่ยังไม่ได้กำหนด schema ตายตัว หรือรอการประมวลผลภายหลัง (log, sensor data, ข้อมูลดิบยังไม่ตรวจสอบ)
- Object Storage เหมาะกับไฟล์ binary ขนาดใหญ่ที่ไม่มีโครงสร้างเชิงตาราง (ภาพ วิดีโอ เสียง PDF)
- Data Warehouse เหมาะกับข้อมูลผ่านการประมวลผล/สรุปแล้ว เพื่อการวิเคราะห์และออกรายงานเชิงธุรกิจ

Lab 2: ทำความสะอาดข้อมูลดิบจากเซนเซอร์ IoT

วัตถุประสงค์: ตรวจสอบและแก้ไขปัญหา Error, Duplicate, Missing Value และ Outlier ในข้อมูลดิบจากเซนเซอร์ IoT (iot_sensor_raw_data_extended.csv) ด้วย Python/Pandas

1. ปัญหาคุณภาพข้อมูลที่พบในแต่ละแถว (สรุป)

จากการตรวจสอบ ชุดข้อมูลดิบ (480 แถว) พบปัญหาคุณภาพข้อมูลดังนี้
(นับซ้ำได้ในแถวเดียวกันหากมีหลายปัญหาพร้อมกัน เช่น แถวหนึ่งอาจทั้งซ้ำและมี temp outlier):

รายการปัญหา	จำนวนที่พบ (แถว)
Duplicate rows (device_id + timestamp ซ้ำ, รวมทั้งชุด)	50
Temp outlier (นอกช่วง 0–50 °C)	10
Battery outlier (นอกช่วง 0–100 %)	5
Missing device_id	2
Missing temp_c	7
Missing battery	4
Motion ค่าไม่ถูกต้อง (ไม่ใช่ true/false)	5
Motion ว่าง (missing)	4

2. Data Quality Rules ที่กำหนด

รหัส	Data Quality Rule	วิธีการจัดการเมื่อผิดกฎ
R1	temp_c ต้องอยู่ระหว่าง 0-50 องศาเซลเซียส	ค่านอกช่วง → แปลงเป็น NULL แล้ว interpolate ตามลำดับเวลาในแต่ละ device
R2	device_id ห้ามเป็นค่าว่าง	แถวที่ device_id ว่าง ไม่สามารถระบุที่มาได้ → ทำเครื่องหมาย invalid_missing_device_id และไม่รวมในชุดข้อมูลสะอาดสุดท้าย (แต่ยังเก็บ log ไว้)
R3	ห้ามมีข้อมูลซ้ำที่มี device_id และ timestamp เดียวกัน	ลบด้วย drop_duplicates(subset=[timestamp, device_id], keep='first')
R4	battery ต้องอยู่ระหว่าง 0-100	ค่านอกช่วง → แปลงเป็น NULL แล้ว interpolate ตาม device
R5	motion ต้องเป็น true/false เท่านั้น	ค่าอื่น → แปลงเป็น NULL ก่อน แล้วเติมด้วยค่าฐานนิยม (mode) ของ device นั้น

3. โปรแกรม Python/Pandas สำหรับทำความสะอาดข้อมูล

ขั้นตอนการทำงานของโปรแกรม: (1) โหลด raw data เก็บต้นฉบับไว้เสมอ (2)

ตรวจนับปัญหาทำความสะอาดเพื่อทำรายงาน (3) ลบข้อมูลซ้ำ (4) แปลงค่าผิดปกติ/error เป็น NULL ตามกฎ R1, R4, R5 (5) เติม Missing Value ด้วยวิธีที่เหมาะสมกับลักษณะข้อมูลแต่ละคอลัมน์ (6) เพิ่มคอลัมน์ data_quality_status (7) บันทึกไฟล์ผลลัพธ์ทั้งแบบที่ยังมีแถวถูกติง (flagged) และแบบสะอาดสุดท้าย:

หมายเหตุวิธีเติม Missing Value:

- temp_c และ battery: ใช้ interpolate() ตามลำดับเวลาในแต่ละ device_id เนื่องจากเป็นค่าต่อเนื่อง (time-series) เปลี่ยนแปลงแบบค่อยเป็นค่อยไป หากยังว่างอยู่ (device มีค่าว่างทั้งหมด) จะเติมด้วยค่ามัธยฐาน (median) รวม
- motion: เป็นค่าหมวดหมู่ (categorical: true/false) จึงเติมด้วยค่าฐานนิยม (mode) ของ device นั้น ๆ
- device_id: ไม่เติมค่า เนื่องจากไม่มีทางทราบได้อย่างน่าเชื่อถือว่าแถวนั้นมาจากเซนเซอร์ตัวใด จึงทำเครื่องหมายเป็น invalid_missing_device_id แทน

4. เปรียบเทียบจำนวนแถวก่อนและหลังทำความสะอาด

ขั้นตอน	จำนวนแถว
ก่อนทำความสะอาด (Raw Data)	480
Duplicate ที่ถูกลบออก	25
หลังลบ Duplicate	455
ถูกตัดออกเพราะ device_id ว่าง (invalid — เก็บ log ไว้ ไม่ลบทิ้ง)	2
ชุดข้อมูลสะอาดสุดท้าย (cleaned_ok, พร้อมใช้งาน)	453

สรุป: จากข้อมูลดิบ 480 แถว หลังลบข้อมูลซ้ำ 25 แถว เหลือ 455 แถว จากนั้นตัดแถวที่ device_id ว่าง 2 แถว (ซึ่งไม่นำไปใช้งานต่อแต่ยังเก็บ log ไว้) คงเหลือชุดข้อมูลสะอาดพร้อมใช้งานสุดท้าย 453 แถว

5. เหตุผลที่ไม่ควรลบไฟล์ Raw Data ต้นฉบับ

- Traceability / Audit: Raw Data คือหลักฐานต้นทางที่ใช้ตรวจสอบย้อนกลับได้ว่าค่าที่ถูกแก้ไขหรือ Data Quality Status ที่กำหนดนั้นถูกต้องหรือไม่ หากลบทิ้งจะไม่สามารถตรวจสอบความถูกต้องของกระบวนการทำความสะอาดได้อีก
- Reproducibility: หากกฎการทำความสะอาด (Data Quality Rules) เปลี่ยนแปลงในอนาคต หรือพบว่าตรรกะการทำความสะอาดผิดพลาด จำเป็นต้องมีข้อมูลดิบไว้เพื่อรันกระบวนการทำความสะอาดใหม่ได้ โดยไม่ต้องรอเก็บข้อมูลใหม่
- ป้องกันการสูญเสียข้อมูล (Data Loss Prevention): ขั้นตอนทำความสะอาดบางอย่าง เช่น การแปลงค่า Outlier เป็น NULL หรือการลบแถวซ้ำ เป็นการ 'สูญเสียข้อมูลบางส่วนอย่างถาวร' หากไม่มีต้นฉบับสำรองไว้ จะไม่สามารถกู้คืนค่าที่แท้จริงกลับมาได้
- การวิเคราะห์เชิงคุณภาพข้อมูล: การเปรียบเทียบ Raw กับ Cleaned Data ช่วยให้เห็นคุณภาพของแหล่งข้อมูลต้นทาง (เช่น อัตรา error ของเซนเซอร์แต่ละตัว) ซึ่งเป็นประโยชน์ต่อการปรับปรุงอุปกรณ์หรือกระบวนการเก็บข้อมูลในอนาคต
- ข้อกำหนดด้าน Compliance: หลายองค์กร/มาตรฐาน (เช่น การตรวจสอบภายใน หรือกฎหมายคุ้มครองข้อมูล) กำหนดให้ต้องเก็บข้อมูลต้นฉบับไว้เป็นระยะเวลาหนึ่งเพื่อการตรวจสอบ

6. ตารางสรุป Error, Duplicate, Missing Value และ Outlier (สรุปรวม)

รายการปัญหา	จำนวนที่พบ (แถว)
Duplicate rows (device_id + timestamp ซ้ำ, รวมทั้งชุด)	50
Temp outlier (นอกช่วง 0–50 °C)	10
Battery outlier (นอกช่วง 0–100 %)	5
Missing device_id	2
Missing temp_c	7
Missing battery	4
Motion ค่าไม่ถูกต้อง (ไม่ใช่ true/false)	5
Motion ว่าง (missing)	4

7. ผลลัพธ์ที่ส่งมอบ

- Source code: สคริปต์ Python/Pandas (Google Colab notebook 67160362.ipynb)
ที่ใช้ทำความสะอาดข้อมูล
- ไฟล์ข้อมูล Cleaned Data: `iot_sensor_cleaned.csv` (453 แถว, cleaned_ok ทั้งหมด) และ `iot_sensor_cleaned_with_flagged_rows.csv` (455 แถว รวมแถวที่ถูกตีธง invalid_missing_device_id ไว้ด้วยเพื่อการตรวจสอบย้อนหลัง)